Introduction:

Ordinary Differential Equations (ODEs) arise naturally across science and engineering — from population dynamics and chemical reactions to electrical circuits and astrophysics. While many ODEs lack closed-form solutions, numerical methods allow us to approximate them to high accuracy. In this project, we investigate eight numerical solvers and analyze their behavior across several key problem settings.

Numerical Methods:

We will examine methods spanning three levels of complexity. The Euler and Taylor 2nd-order methods are simple single-step schemes, with Taylor gaining accuracy as it uses derivative information to shrink that error. The Midpoint, Heun's, and RK4 methods are Runge-Kutta-type solvers that evaluate the right-hand side at intermediate points, with RK4 serving as the gold standard for single-step integration. Finally, Adams-Bashforth, Adams-Moulton, and the Predictor-Corrector are multi-step methods that achieve high accuracy with fewer function evaluations per step.

In [ ]:
```python
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

In [ ]:
```python
# Euler method
def euler(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    for i in range(N):
        w[i + 1] = w[i] + h * f(t[i], w[i])
    return t, w
```

In [ ]:
```python
# Taylor 2nd-order method
def taylor2(f, f_prime, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    for i in range(N):
        T_2 = f(t[i], w[i]) + (h / 2) * f_prime(t[i], w[i])
        w[i + 1] = w[i] + h * T_2
    return t, w
```

In [ ]:
```python
# Midpoint method
```

```
def midpoint(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    for i in range(N):
        w[i + 1] = w[i] + h * f(t[i] + h / 2, w[i] + (h / 2) * f(t[i],
    return t, w
```

In [ ]:
```
# Heun's 3rd-order method
def heuns(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    for i in range(N):
        w[i + 1] = w[i] + (h / 4) * (
            f(t[i], w[i])
            + 3 * f(
                t[i] + (2 * h) / 3,
                w[i] + (2 * h / 3) * f(t[i] + h / 3, w[i] + (h / 3) *
            )
        )
    return t, w
```

In [ ]:
```
# Runge-Kutta 4th-order (RK4)
def RK4(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    for i in range(N):
        k1 = h * f(t[i], w[i])
        k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
        k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
        k4 = h * f(t[i + 1], w[i] + k3)
        w[i + 1] = w[i] + (k1 + 2 * k2 + 2 * k3 + k4) / 6
    return t, w
```

In [ ]:
```
# Adams-Bashforth 4-step explicit method
def adams_bashforth(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    # Bootstrap first 3 steps with RK4
    for i in range(3):
        k1 = h * f(t[i], w[i])
        k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
        k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
        k4 = h * f(t[i] + h, w[i] + k3)
        w[i + 1] = w[i] + (k1 + 2 * k2 + 2 * k3 + k4) / 6
```

```
    # Adams-Bashforth 4-step formula
    for i in range(3, N):
        w[i + 1] = w[i] + (h / 24) * (
            55 * f(t[i],     w[i])
          - 59 * f(t[i - 1], w[i - 1])
          + 37 * f(t[i - 2], w[i - 2])
          -  9 * f(t[i - 3], w[i - 3])
        )
    return t, w
```

In [ ]:
```
# Adams-Moulton 3-step implicit method
def adams_moulten(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha

    # 2 steps with RK4
    for i in range(2):
        k1 = h * f(t[i], w[i])
        k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
        k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
        k4 = h * f(t[i] + h, w[i] + k3)
        w[i + 1] = w[i] + (k1 + 2 * k2 + 2 * k3 + k4) / 6

    for i in range(2, N):
        # Predictor (Adams-Bashforth 3-step)
        w_pred = w[i] + (h / 12) * (
            23 * f(t[i],     w[i])
          - 16 * f(t[i - 1], w[i - 1])
          +  5 * f(t[i - 2], w[i - 2])
        )
        # Corrector (Adams-Moulton 3-step)
        w[i + 1] = w[i] + (h / 24) * (
             9 * f(t[i + 1], w_pred)
          + 19 * f(t[i],     w[i])
          -  5 * f(t[i - 1], w[i - 1])
          +      f(t[i - 2], w[i - 2])
        )
    return t, w
```

In [ ]:
```
def predictor_corrector(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w   = np.zeros(N + 1)
    w_p = np.zeros(N + 1)
    w[0] = w_p[0] = alpha

    # First 3 steps with RK4
    for i in range(3):
        k1 = h * f(t[i], w[i])
        k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
```

```
        k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
        k4 = h * f(t[i] + h, w[i] + k3)
        w[i + 1] = w_p[i + 1] = w[i] + (k1 + 2*k2 + 2*k3 + k4) / 6

    # Adams-Bashforth 4-step predictor + Adams-Moulton 4-step correcto
    for i in range(3, N):
        # Predictor
        w_p[i + 1] = w[i] + (h / 24) * (
              55 * f(t[i],     w[i])
            - 59 * f(t[i - 1], w[i - 1])
            + 37 * f(t[i - 2], w[i - 2])
            -  9 * f(t[i - 3], w[i - 3])
        )
        # Corrector
        w[i + 1] = w[i] + (h / 24) * (
              9 * f(t[i + 1], w_p[i + 1])
            + 19 * f(t[i],     w[i])
            -  5 * f(t[i - 1], w[i - 1])
            +      f(t[i - 2], w[i - 2])
        )

    return t, w
```

Problem A)

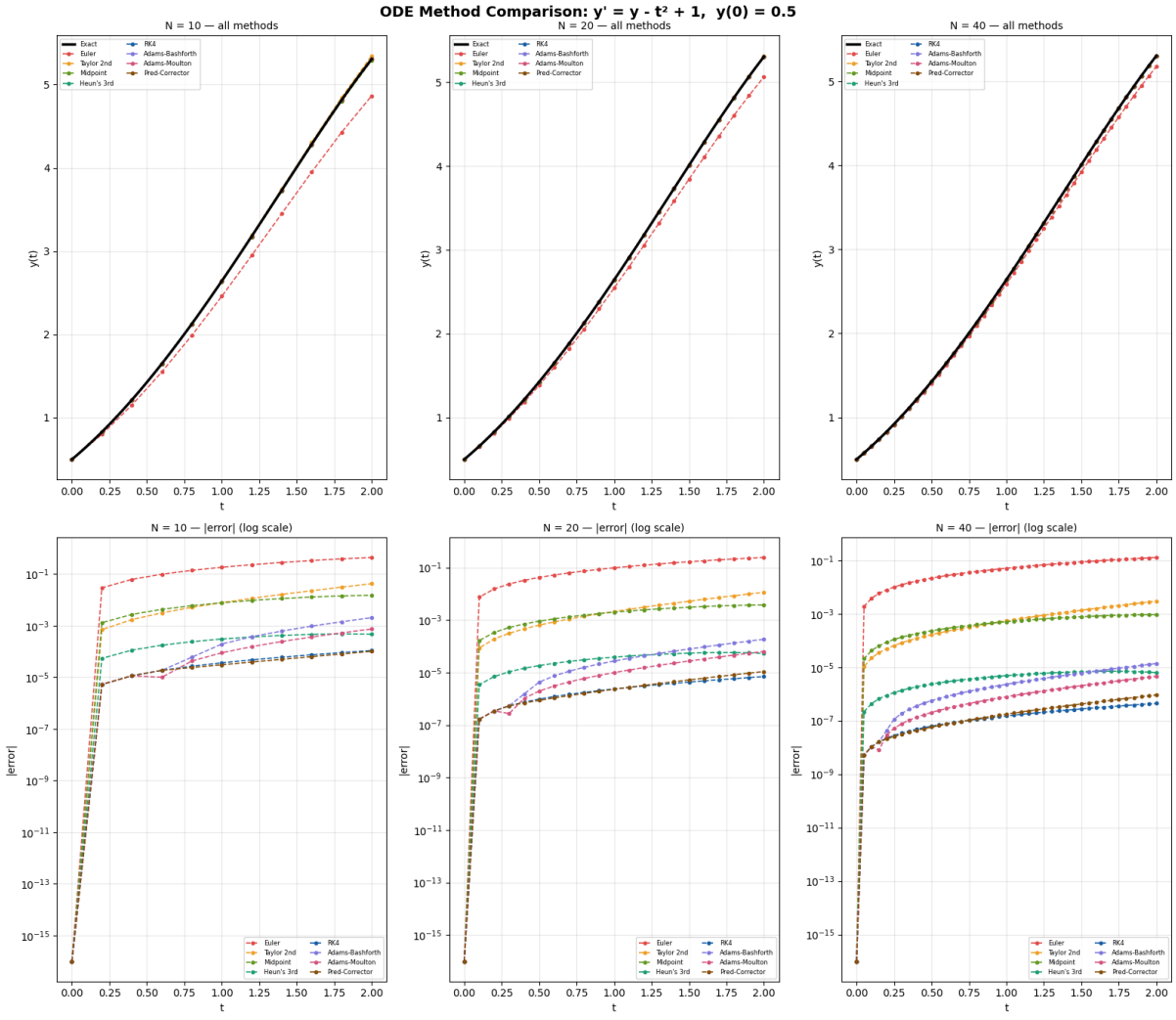
In [ ]:

```
# ODE: y' = y - t^2 + 1,  y(0) = 1
# Exact solution: y(t) = (t+1)^2 - 0.5 * exp(t)

def f_1(t, y):
    return y - t**2 + 1

def f_1_p(t, y):
    return y - t**2 - 2*t + 1


def y_exact(t):
    return (t + 1)**2 - 0.5 * np.exp(t)


N_s = [10, 20, 40]

results = {}
for N in N_s:
    results[N] = {
        'euler':     euler(f_1, 0, 2, N, alpha=0.5),
        'taylor':    taylor2(f_1, f_1_p, 0, 2, N, alpha=0.5),
        'midpoint':  midpoint(f_1, 0, 2, N, alpha=0.5),
        'heuns':     heuns(f_1, 0, 2, N, alpha=0.5),
        'RK4':       RK4(f_1, 0, 2, N, alpha=0.5),
        'adams_b':   adams_bashforth(f_1, 0, 2, N, alpha=0.5),
        'adams_m':   adams_moulten(f_1, 0, 2, N, alpha=0.5),
```

```
        'pred_cor':  predictor_corrector(f_1, 0, 2, N, alpha=0.5),
    }
```

In [ ]:

```
method_keys   = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams
method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4
                 'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

fig, axes = plt.subplots(2, 3, figsize=(16, 14))
fig.suptitle("ODE Method Comparison: y' = y − t² + 1,  y(0) = 0.5", fo

colors = ['#E24B4A', '#EF9F27', '#639922', '#1D9E75', '#185FA5', '#7F7

for col, N in enumerate(N_s):
    t_fine = np.linspace(0, 2, 300)

    ax_top = axes[0][col]
    ax_top.plot(t_fine, y_exact(t_fine), 'k-', linewidth=2.5, label='E
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results[N][key]
        ax_top.plot(t_m, w_m, '--o', color=color, markersize=3, linewi
    ax_top.set_title(f'N = {N} − all methods', fontsize=10)
    ax_top.set_xlabel('t')
    ax_top.set_ylabel('y(t)')
    ax_top.legend(fontsize=6, ncol=2)
    ax_top.grid(True, alpha=0.3)


    ax_mid = axes[1][col]
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results[N][key]
        err = np.abs(w_m - y_exact(t_m))
        ax_mid.semilogy(t_m, err + 1e-16, '--o', color=color, markersi
    ax_mid.set_title(f'N = {N} − |error| (log scale)', fontsize=10)
    ax_mid.set_xlabel('t')
    ax_mid.set_ylabel('|error|')
    ax_mid.legend(fontsize=6, ncol=2)
    ax_mid.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



In [ ]:

```
%%capture
for N in N_s:
    t_vals = results[N]['euler'][0]   # all methods share the same t
    exact_vals = y_exact(t_vals)

    data = {'t': t_vals, 'Exact': exact_vals}
    for key, label in zip(method_keys, method_labels):
        _, w = results[N][key]
        data[label] = w

    df = pd.DataFrame(data)

    max_row = {'t': 'max |error|', 'Exact': ''}
    for key, label in zip(method_keys, method_labels):
        _, w = results[N][key]
        max_row[label] = np.max(np.abs(w - y_exact(t_vals)))
    df = pd.concat([df, pd.DataFrame([max_row])], ignore_index=True)

    print(f'\n{"="*60}')
    print(f'  N = {N}  (h = {2/N:.4f})')
    print(f'{"="*60}')
```

```
    float_cols = [c for c in df.columns if c != 't']
    pd.set_option('display.float_format', '{:.8f}'.format)
    pd.set_option('display.max_columns', 20)
    pd.set_option('display.width', 220)
    print(df.to_string(index=False))
```

In [ ]:

```
%%capture
import pandas as pd

method_keys   = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams
method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4
                 'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

for N in N_s:
    h = 2 / N
    t_vals     = results[N]['euler'][0]
    exact_vals = y_exact(t_vals)

    data = {'t': np.round(t_vals, 4), 'Exact': exact_vals}
    for key, label in zip(method_keys, method_labels):
        _, w = results[N][key]
        data[label] = np.abs(w - exact_vals)

    df = pd.DataFrame(data)

    # format: exact in regular float, errors in scientific notation
    fmt = {'Exact': '{:.6f}'.format}
    for label in method_labels:
        fmt[label] = '{:.2e}'.format

    print(f'\nError table  |  h = {h}  (N = {N})\n')
    print(df.to_string(index=False, formatters=fmt))
    print()
```

Problem A uses the ODE $y' = y - t^2 + 1$ with exact solution $y(t) = (t+1)^2 - \frac{1}{2}e^t$. Because the solution is smooth and non-stiff, it provides an excellent test of convergence behavior. The numerical results clearly reflect the theoretical orders of the methods. At t=2, Euler's error decreases from approximately $4.40 * e^{-1}$ for N=10 to $2.42 * e^{-1}$ for N=20, and then to roughly $1.27 * e^{-1}$ for N=40, demonstrating first-order convergence. The second-order methods (Taylor and Midpoint) show much faster improvement. Taylor's error falls from approximately $4.22 * e^{-02}$ to $2.96 * e^{-03}$, while the Midpoint method decreases from $1.51 * e^{-02}$ to roughly $9.28 * e^{-04}$ .Higher-order methods perform dramatically better as expected. Heun's method, RK4, Adams-Moulton, and the Predictor-Corrector method all produce errors several orders of magnitude smaller than Euler's method. In the plots, the RK4 and multistep solutions are nearly indistinguishable from the exact solution, even with relatively coarse step sizes. As the step size decreases, the errors consistently decrease at

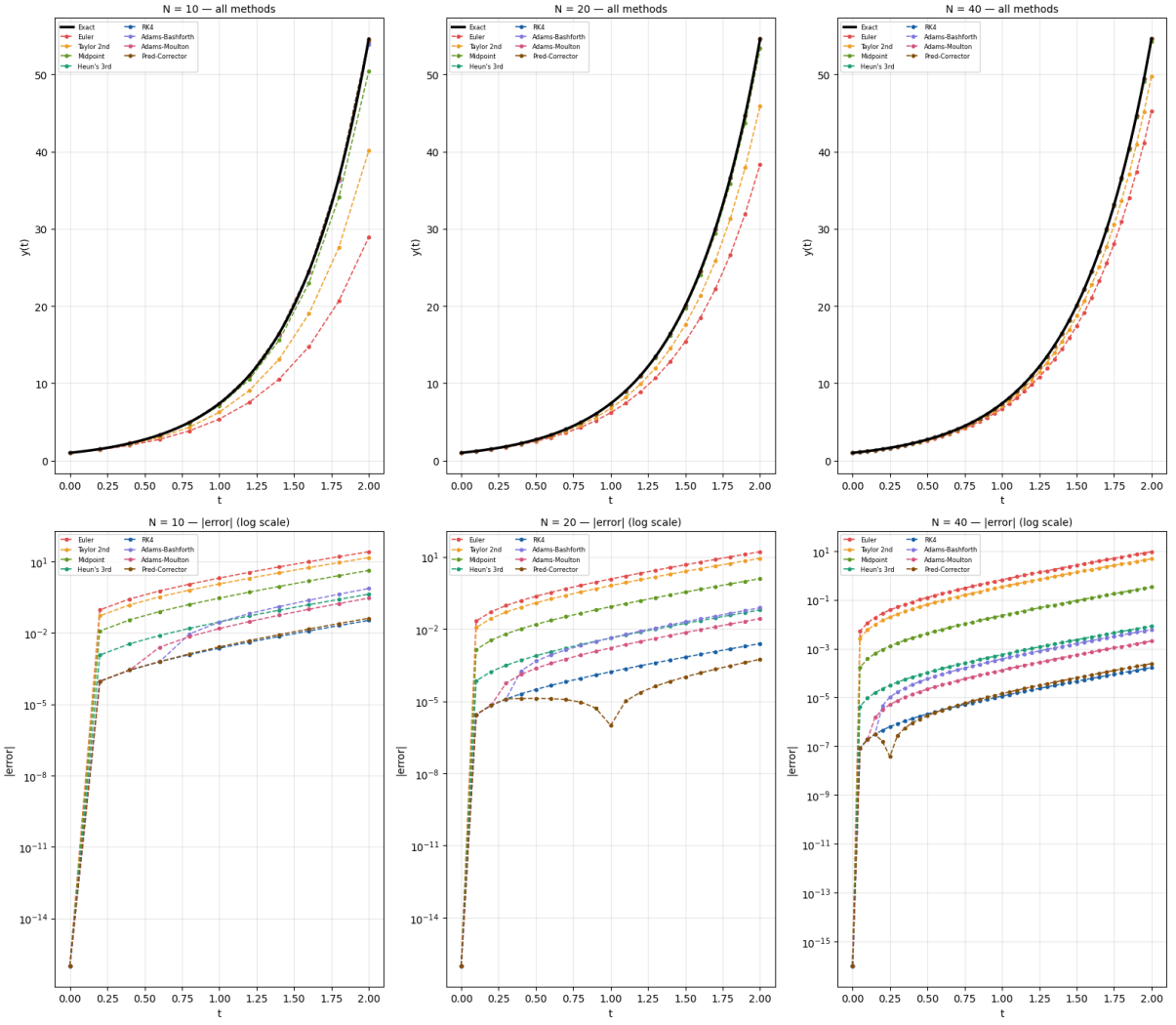
rates predicted by the theoretical convergence orders, confirming the expected relationship between local truncation error and global error.

Problem B)

In [ ]:
```
def f_2(t, y):
    return 2*y

def y_exact_2(t):
    return np.exp(2*t)

def f_2_p(t,y):
    return 2 * np.exp(2*t)

N_s = [10, 20, 40]

results_2 = {}
for N in N_s:
    results_2[N] = {
        'euler':     euler(f_2, 0, 2, N, alpha=1),
        'taylor':    taylor2(f_2, f_2_p, 0, 2, N, alpha=1),
        'midpoint':  midpoint(f_2, 0, 2, N, alpha=1),
        'heuns':     heuns(f_2, 0, 2, N, alpha=1),
        'RK4':       RK4(f_2, 0, 2, N, alpha=1),
        'adams_b':   adams_bashforth(f_2, 0, 2, N, alpha=1),
        'adams_m':   adams_moulten(f_2, 0, 2, N, alpha=1),
        'pred_cor':  predictor_corrector(f_2, 0, 2, N, alpha=1),
    }
```

In [ ]:
```
fig, axes = plt.subplots(2, 3, figsize=(16, 14))

for col, N in enumerate(N_s):
    t_fine = np.linspace(0, 2, 300)
    ax_top = axes[0][col]
    ax_top.plot(t_fine, y_exact_2(t_fine), 'k-', linewidth=2.5, label=
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results_2[N][key]
        ax_top.plot(t_m, w_m, '--o', color=color, markersize=3, linewi
    ax_top.set_title(f'N = {N} — all methods', fontsize=10)
    ax_top.set_xlabel('t')
    ax_top.set_ylabel('y(t)')
    ax_top.legend(fontsize=6, ncol=2)
    ax_top.grid(True, alpha=0.3)

    ax_mid = axes[1][col]
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results_2[N][key]
        err = np.abs(w_m - y_exact_2(t_m))   # ← fixed
        ax_mid.semilogy(t_m, err + 1e-16, '--o', color=color, markersi
    ax_mid.set_title(f'N = {N} — |error| (log scale)', fontsize=10)
    ax_mid.set_xlabel('t')
```

```
    ax_mid.set_ylabel('|error|')
    ax_mid.legend(fontsize=6, ncol=2)
    ax_mid.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



In [ ]:
```
%%capture
for N in N_s:

    t_vals = results_2[N]['euler'][0]   # all methods share the same t
    exact_vals_2 = y_exact_2(t_vals)

    data = {'t': t_vals, 'Exact': exact_vals_2}
    for key, label in zip(method_keys, method_labels):
        _, w = results_2[N][key]
        data[label] = w

    df = pd.DataFrame(data)

    max_row = {'t': 'max |error|', 'Exact': ''}
    for key, label in zip(method_keys, method_labels):
        _, w = results_2[N][key]
```

```
        max_row[label] = np.max(np.abs(w - y_exact_2(t_vals)))
    df = pd.concat([df, pd.DataFrame([max_row])], ignore_index=True)

    print(f'\n{"="*60}')
    print(f'  N = {N}  (h = {2/N:.4f})')
    print(f'{"="*60}')


    float_cols = [c for c in df.columns if c != 't']
    pd.set_option('display.float_format', '{:.8f}'.format)
    pd.set_option('display.max_columns', 20)
    pd.set_option('display.width', 220)
    print(df.to_string(index=False))
```

In [ ]:

```
%%capture
method_keys   = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams
method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4
                 'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

for N in N_s:
    h = 2 / N
    t_vals     = results_2[N]['euler'][0]
    exact_vals_2 = y_exact_2(t_vals)

    data = {'t': np.round(t_vals, 4), 'Exact': exact_vals_2}
    for key, label in zip(method_keys, method_labels):
        _, w = results_2[N][key]
        data[label] = np.abs(w - exact_vals_2)

    df = pd.DataFrame(data)

    # format: exact in regular float, errors in scientific notation
    fmt = {'Exact': '{:.6f}'.format}
    for label in method_labels:
        fmt[label] = '{:.2e}'.format

    print(f'\nError table  |  h = {h}  (N = {N})\n')
    print(df.to_string(index=False, formatters=fmt))
    print()
```

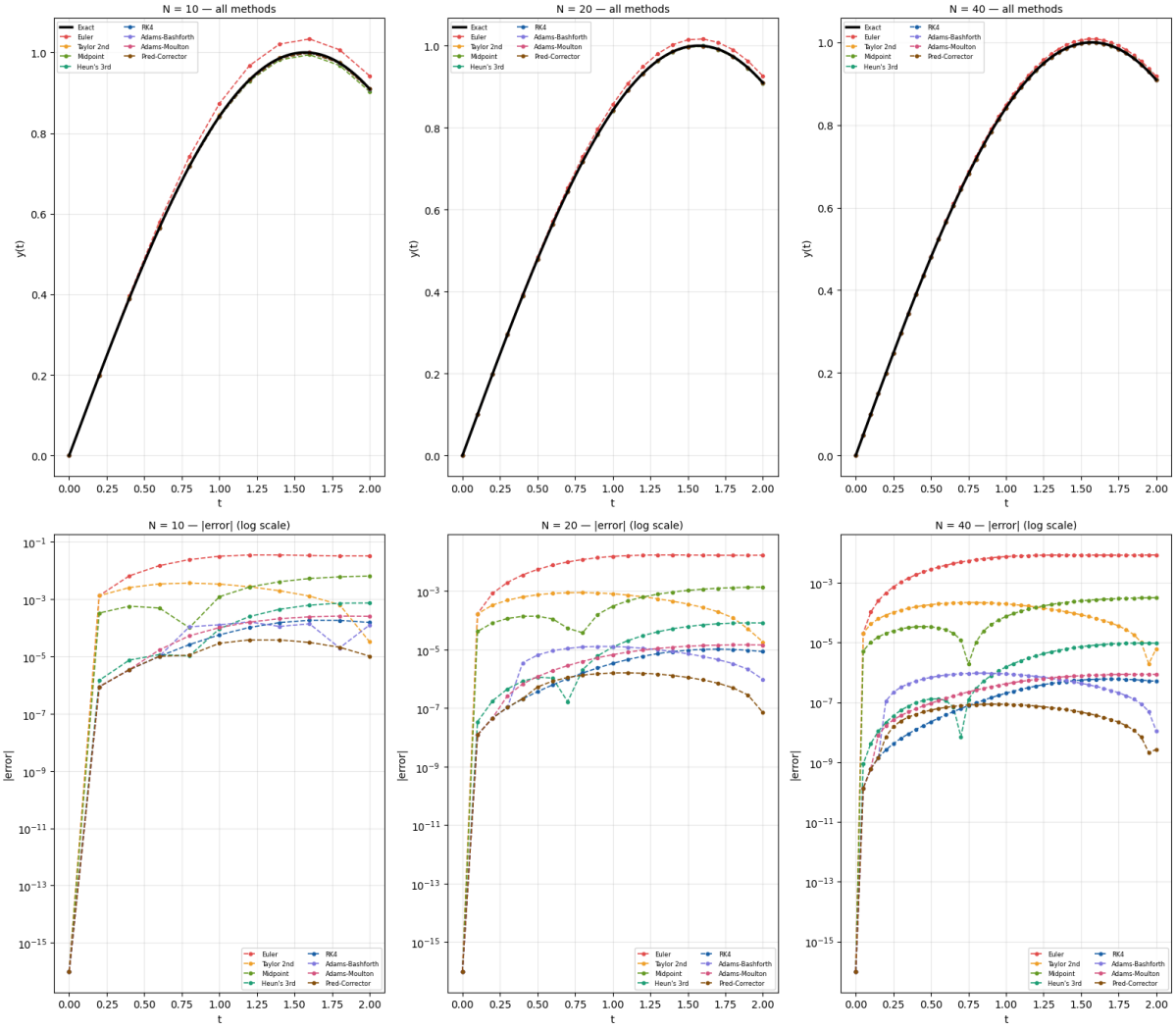
Problem B investigates the exponentially growing solution $y = e^{2t}$. This problem is particularly useful for studying error amplification because any numerical error introduced during the computation is magnified by the rapid growth of the exact solution. The results show that lower-order methods struggle significantly with this behavior. For N=10, Euler's maximum error reaches approximately 25.67, which is a substantial fraction of the exact solution value at $t = 2$. The second-order Taylor method reduces the maximum error to about 14.49, while the Midpoint method further lowers it to approximately 4.18. Increasing the number of steps improves the accuracy of all methods, but the higher-order methods

remain substantially more accurate. For $N = 20$, RK4 achieves a maximum error of only 0.00246581, compared with 16.26055011 for Euler. At N=40, RK4's maximum error drops to approximately 1.67×10^{-4}, while Euler still exhibits an error greater than 9.33. Adams-Moulton and the Predictor-Corrector method also perform exceptionally well, maintaining errors on the order of 10^{-3} or smaller. The logarithmic error plots illustrate how truncation errors are amplified over time by the exponential growth of the solution. These results demonstrate the importance of both method order and stability when solving problems in which errors naturally grow throughout the integration interval.

Problem C)

In [ ]:
```
def f_3(t, y):
    return -y**3 + np.cos(t) + np.sin(t)**3

def f_3_p(t, y):
    return -np.sin(t) + 3*np.sin(t)**2 * np.cos(t) + (-3*y**2) * (-y**

def y_exact_3(t):
    return np.sin(t)

results_3 = {}
for N in N_s:
    results_3[N] = {
        'euler':     euler(f_3, 0, 2, N, alpha=0),
        'taylor':    taylor2(f_3, f_3_p, 0, 2, N, alpha=0),
        'midpoint':  midpoint(f_3, 0, 2, N, alpha=0),
        'heuns':     heuns(f_3, 0, 2, N, alpha=0),
        'RK4':       RK4(f_3, 0, 2, N, alpha=0),
        'adams_b':   adams_bashforth(f_3, 0, 2, N, alpha=0),
        'adams_m':   adams_moulten(f_3, 0, 2, N, alpha=0),
        'pred_cor':  predictor_corrector(f_3, 0, 2, N, alpha=0),
    }

fig, axes = plt.subplots(2, 3, figsize=(16, 14))
for col, N in enumerate(N_s):
    t_fine = np.linspace(0, 2, 300)
    ax_top = axes[0][col]
    ax_top.plot(t_fine, y_exact_3(t_fine), 'k-', linewidth=2.5, label=
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results_3[N][key]
        ax_top.plot(t_m, w_m, '--o', color=color, markersize=3, linewi
    ax_top.set_title(f'N = {N} — all methods', fontsize=10)
    ax_top.set_xlabel('t')
    ax_top.set_ylabel('y(t)')
    ax_top.legend(fontsize=6, ncol=2)
    ax_top.grid(True, alpha=0.3)

    ax_mid = axes[1][col]
```

```
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results_3[N][key]
        err = np.abs(w_m - y_exact_3(t_m))
        ax_mid.semilogy(t_m, err + 1e-16, '--o', color=color, markersi
    ax_mid.set_title(f'N = {N} — |error| (log scale)', fontsize=10)
    ax_mid.set_xlabel('t')
    ax_mid.set_ylabel('|error|')
    ax_mid.legend(fontsize=6, ncol=2)
    ax_mid.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



In [ ]:

```
%%capture
for N in N_s:

    t_vals = results_3[N]['euler'][0]   # all methods share the same t
    exact_vals_3 = y_exact_3(t_vals)

    data = {'t': t_vals, 'Exact': exact_vals_3}
    for key, label in zip(method_keys, method_labels):
        _, w = results_3[N][key]
        data[label] = w
```

```
    df = pd.DataFrame(data)

    max_row = {'t': 'max |error|', 'Exact': ''}
    for key, label in zip(method_keys, method_labels):
        _, w = results_3[N][key]
        max_row[label] = np.max(np.abs(w - y_exact_3(t_vals)))
    df = pd.concat([df, pd.DataFrame([max_row])], ignore_index=True)

    print(f'\n{"="*60}')
    print(f'  N = {N}  (h = {2/N:.4f})')
    print(f'{"="*60}')


    float_cols = [c for c in df.columns if c != 't']
    pd.set_option('display.float_format', '{:.8f}'.format)
    pd.set_option('display.max_columns', 20)
    pd.set_option('display.width', 220)
    print(df.to_string(index=False))
```

In [ ]:

```
%%capture
method_keys   = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams
method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4
                 'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

for N in N_s:
    h = 2 / N
    t_vals     = results_3[N]['euler'][0]
    exact_vals_3 = y_exact_3(t_vals)

    data = {'t': np.round(t_vals, 4), 'Exact': exact_vals_3}
    for key, label in zip(method_keys, method_labels):
        _, w = results_3[N][key]
        data[label] = np.abs(w - exact_vals_3)

    df = pd.DataFrame(data)

    # format: exact in regular float, errors in scientific notation
    fmt = {'Exact': '{:.6f}'.format}
    for label in method_labels:
        fmt[label] = '{:.2e}'.format

    print(f'\nError table  |  h = {h}  (N = {N})\n')
    print(df.to_string(index=False, formatters=fmt))
    print()
```

Problem C uses the nonlinear differential equation $y' = -y^3 + cos(t) + sin^3(t)$ with exact solution $y(t) = sin(t)$. This problem was selected because it contains a nonlinear term while still having a known analytical solution for comparison. The cubic damping term $-y^3$ helps suppress excessive growth, making the problem more stable than the exponential-growth problem studied in Problem B. As

expected, all methods perform better in this setting. For $N = 10$, Euler's error near the final time remains on the order of 10^{-2}, while Taylor's method reduces the error to approximately 10^-3. The higher-order methods perform significantly better, with RK4, Adams-Moulton, and the Predictor-Corrector method producing errors between 10^{-5} and 10^{-7} across most of the interval. As the step size is reduced from $h = 0.2$ to $h = 0.05$, all methods exhibit the expected decrease in error, with higher-order methods converging much more rapidly than Euler's method. By $N = 40$, RK4 and the Predictor-Corrector method maintain errors that are extremely small and nearly indistinguishable from the exactsolution on the plots. These results show that nonlinear terms do not necessarily create numerical difficulties when stabilizing effects are present. The observed behavior agrees closely with theoretical expectations regarding convergence order, error propagation, and numerical stability, with RK4 and the corrected multistep methods providing the best balance of accuracy and efficiency.

Error Analysis:

Across all three problems, the observed errors agree with theoretical predictions. Euler (order 1) converges slowest, Taylor and Midpoint (order 2) converge moderately, Heun's (order 3) is better still, and RK4 and multi-step methods (order 4) are consistently the most accurate. Methods with the same formal order can still differ due to their leading error constants — for example, Adams-Moulton outperforms Adams-Bashforth because its implicit corrector step reduces the error constant. All methods remained stable within the step sizes tested, though Euler and Taylor would be first to lose stability if h were increased. Error propagation was well-behaved across all problems, with the nonlinear damping in Problem C actually helping to suppress error growth.

Conclusion:

Higher-order methods achieve dramatically better accuracy for the same computational cost. RK4 strikes the best balance for single-step integration, self-starting, stable, and 4th-order accurate. Multi-step methods match RK4's accuracy with fewer evaluations per step, but require carefulness and are more sensitive to initialization errors. The Predictor-Corrector is the most sophisticated method studied, combining explicit and implicit steps to achieve near-implicit accuracy without a nonlinear solve. Correct problem setup — particularly initial conditions — proved just as important as method choice, as an incorrect initial condition rendered all methods inaccurate regardless of their order.